In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from xgboost import XGBClassifier
from sklearn import metrics

import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
df = pd.read_csv("C:\\Users\\karth\\Downloads\\archive (1)\\TSLA.csv")
df.head()
```

Out[2]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	2010-06-29	3.800	5.000	3.508	4.778	4.778	93831500
1	2010-06-30	5.158	6.084	4.660	4.766	4.766	85935500
2	2010-07-01	5.000	5.184	4.054	4.392	4.392	41094000
3	2010-07-02	4.600	4.620	3.742	3.840	3.840	25699000
4	2010-07-06	4.000	4.000	3.166	3.222	3.222	34334500

In [3]:
```
df.shape
```

Out[3]:
```
(2956, 7)
```

In [4]:
```
df.describe()
```

Out[4]:

	Open	High	Low	Close	Adj Close	Volume
count	2956.000000	2956.000000	2956.000000	2956.000000	2956.000000	2.956000e+03
mean	138.691296	141.771603	135.425953	138.762183	138.762183	3.131449e+07
std	250.044839	255.863239	243.774157	250.123115	250.123115	2.798383e+07
min	3.228000	3.326000	2.996000	3.160000	3.160000	5.925000e+05
25%	19.627000	20.402000	19.127500	19.615000	19.615000	1.310288e+07
50%	46.656999	47.487002	45.820002	46.545000	46.545000	2.488680e+07
75%	68.057001	69.357500	66.911501	68.103998	68.103998	3.973875e+07
max	1234.410034	1243.489990	1217.000000	1229.910034	1229.910034	3.046940e+08

In [5]:
```
df.skew()
```

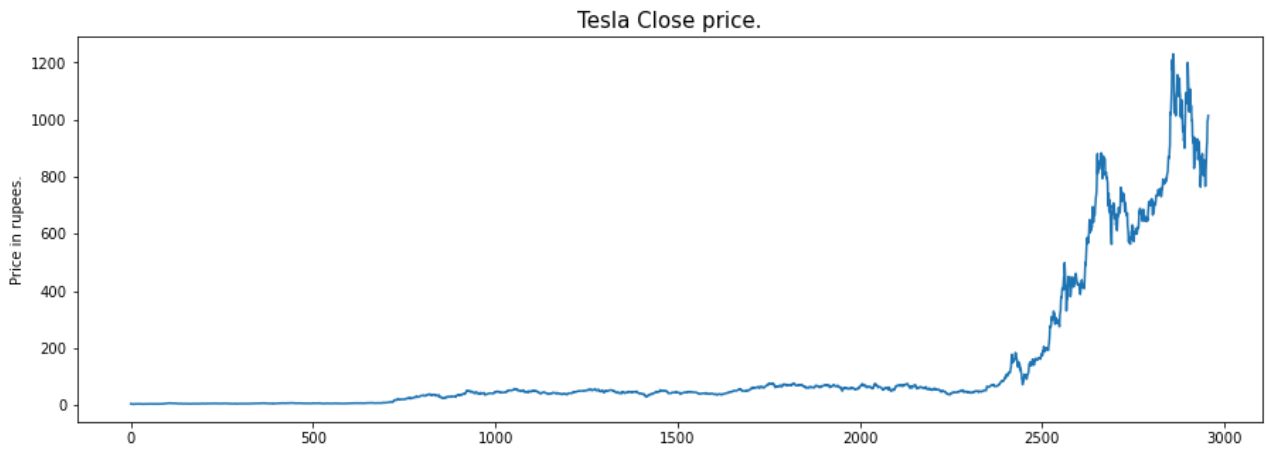
Out[5]:
```
Open          2.399985
High          2.401975
Low           2.395215
```

```
Close        2.397471
Adj Close    2.397471
Volume       2.321703
dtype: float64
```

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2956 entries, 0 to 2955
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Date       2956 non-null   object
 1   Open       2956 non-null   float64
 2   High       2956 non-null   float64
 3   Low        2956 non-null   float64
 4   Close      2956 non-null   float64
 5   Adj Close  2956 non-null   float64
 6   Volume     2956 non-null   int64
dtypes: float64(5), int64(1), object(1)
memory usage: 161.8+ KB
```

In [7]:
```
plt.figure(figsize=(15,5))
plt.plot(df['Close'])
plt.title('Tesla Close price.', fontsize=15)
plt.ylabel('Price in rupees.')
plt.show()
```



In [8]:
```
df.head()
```

Out[8]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	2010-06-29	3.800	5.000	3.508	4.778	4.778	93831500
1	2010-06-30	5.158	6.084	4.660	4.766	4.766	85935500
2	2010-07-01	5.000	5.184	4.054	4.392	4.392	41094000
3	2010-07-02	4.600	4.620	3.742	3.840	3.840	25699000
4	2010-07-06	4.000	4.000	3.166	3.222	3.222	34334500

In [9]:
```
df[df['Close'] == df['Adj Close']].shape
```

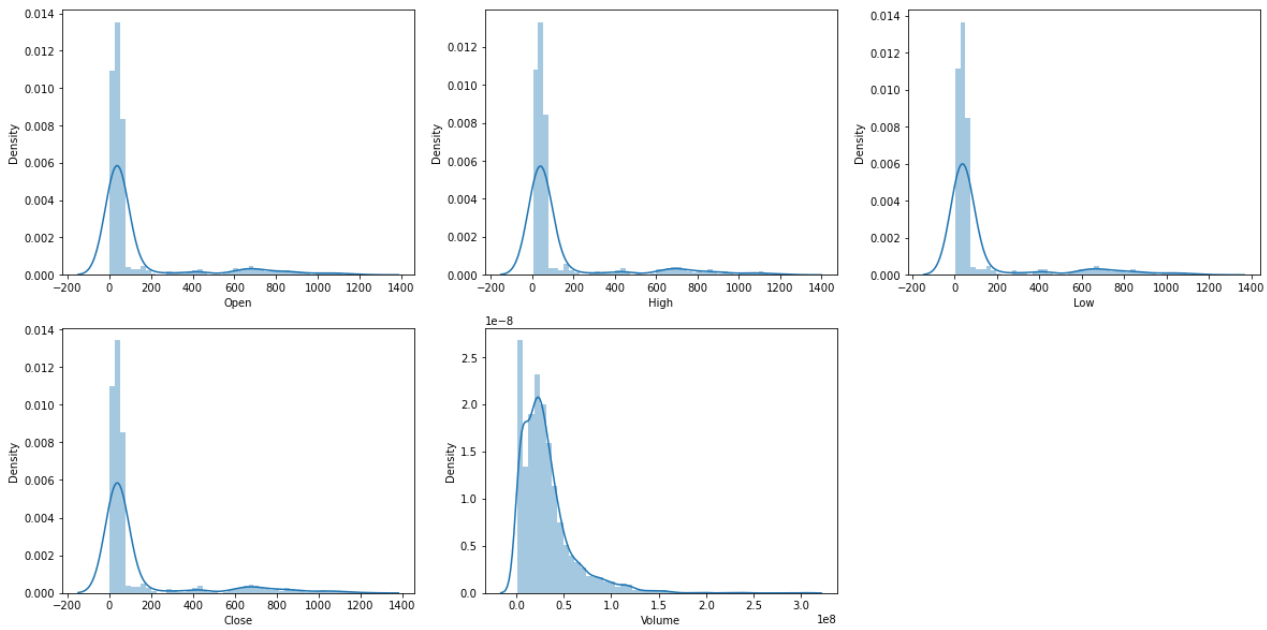
Out[9]:
```
(2956, 7)
```

In [10]:
```
df = df.drop(['Adj Close'], axis=1)
```

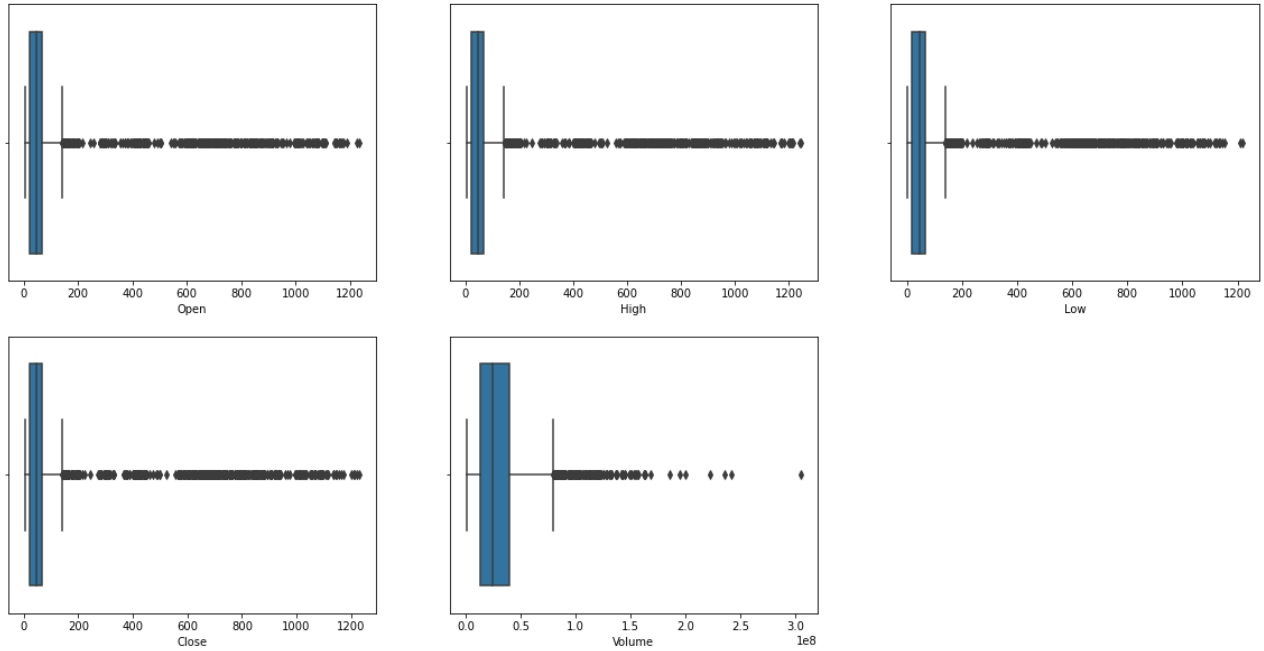
In [11]:
```
df.isnull().sum()
```

Out[11]:
```
Date      0
Open      0
High      0
Low       0
Close     0
Volume    0
dtype: int64
```

In [12]:
```
features = ['Open', 'High', 'Low', 'Close', 'Volume']

plt.subplots(figsize=(20,10))

for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
  sb.distplot(df[col])
plt.show()
```



In [13]:
```
plt.subplots(figsize=(20,10))
for i, col in enumerate(features):
  plt.subplot(2,3,i+1)
  sb.boxplot(df[col])
plt.show()
```



In [14]:
```
splitted = df['Date'].str.split('-', expand=True)

df['day'] = splitted[1].astype('int')
df['month'] = splitted[0].astype('int')
df['year'] = splitted[2].astype('int')

df.head()
```

Out[14]:

	Date	Open	High	Low	Close	Volume	day	month	year
0	2010-06-29	3.800	5.000	3.508	4.778	93831500	6	2010	29
1	2010-06-30	5.158	6.084	4.660	4.766	85935500	6	2010	30
2	2010-07-01	5.000	5.184	4.054	4.392	41094000	7	2010	1
3	2010-07-02	4.600	4.620	3.742	3.840	25699000	7	2010	2
4	2010-07-06	4.000	4.000	3.166	3.222	34334500	7	2010	6

In [15]:
```
df['Date'].head()
```

Out[15]:
```
0    2010-06-29
1    2010-06-30
2    2010-07-01
3    2010-07-02
4    2010-07-06
Name: Date, dtype: object
```

In [16]:
```
df['is_quarter_end'] = np.where(df['month']%3==0,1,0)
df.head()
```

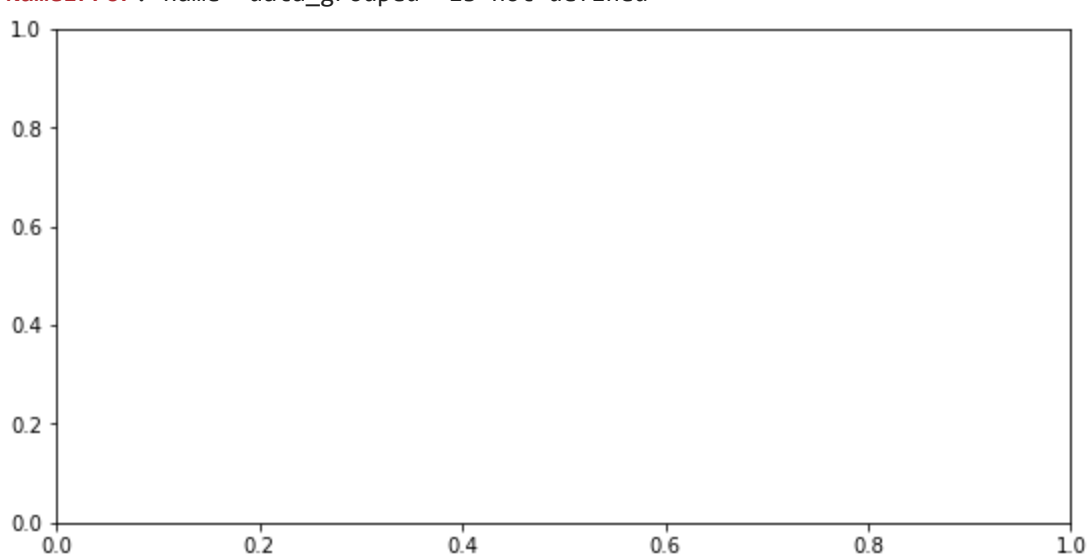
Out[16]:

	Date	Open	High	Low	Close	Volume	day	month	year	is_quarter_end
0	2010-06-29	3.800	5.000	3.508	4.778	93831500	6	2010	29	1
1	2010-06-30	5.158	6.084	4.660	4.766	85935500	6	2010	30	1
2	2010-07-01	5.000	5.184	4.054	4.392	41094000	7	2010	1	1

	Date	Open	High	Low	Close	Volume	day	month	year	is_quarter_end
3	2010-07-02	4.600	4.620	3.742	3.840	25699000	7	2010	2	1
4	2010-07-06	4.000	4.000	3.166	3.222	34334500	7	2010	6	1

In [17]:
```
plt.figure(figsize=(20,10))

for i, col in enumerate(['Open', 'High', 'Low', 'Close']):
    plt.subplot(2,2,i+1)
    plt.bar(data_grouped['year'], data_grouped[col])
    plt.title(col)
    plt.xlabel("Year")
    plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
# yearly comparision is seen here
#so growth and volatility across different periods is highlited
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-17-3e4301a23725> in <module>
      3 for i, col in enumerate(['Open', 'High', 'Low', 'Close']):
      4     plt.subplot(2,2,i+1)
----> 5     plt.bar(data_grouped['year'], data_grouped[col])
      6     plt.title(col)
      7     plt.xlabel("Year")

NameError: name 'data_grouped' is not defined
```



In [18]:
```
data_grouped.head()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-18-7e3fa0b47a62> in <module>
----> 1 data_grouped.head()

NameError: name 'data_grouped' is not defined
```

In [19]:
```
df.drop(['year','month','day'], axis=1, inplace=True)
```

In [20]:
```
df['Date'] = pd.to_datetime(df['Date'])
```

In [21]:
```
df['year'] = df['Date'].dt.year
df['month'] = df['Date'].dt.month
df['day'] = df['Date'].dt.day
```

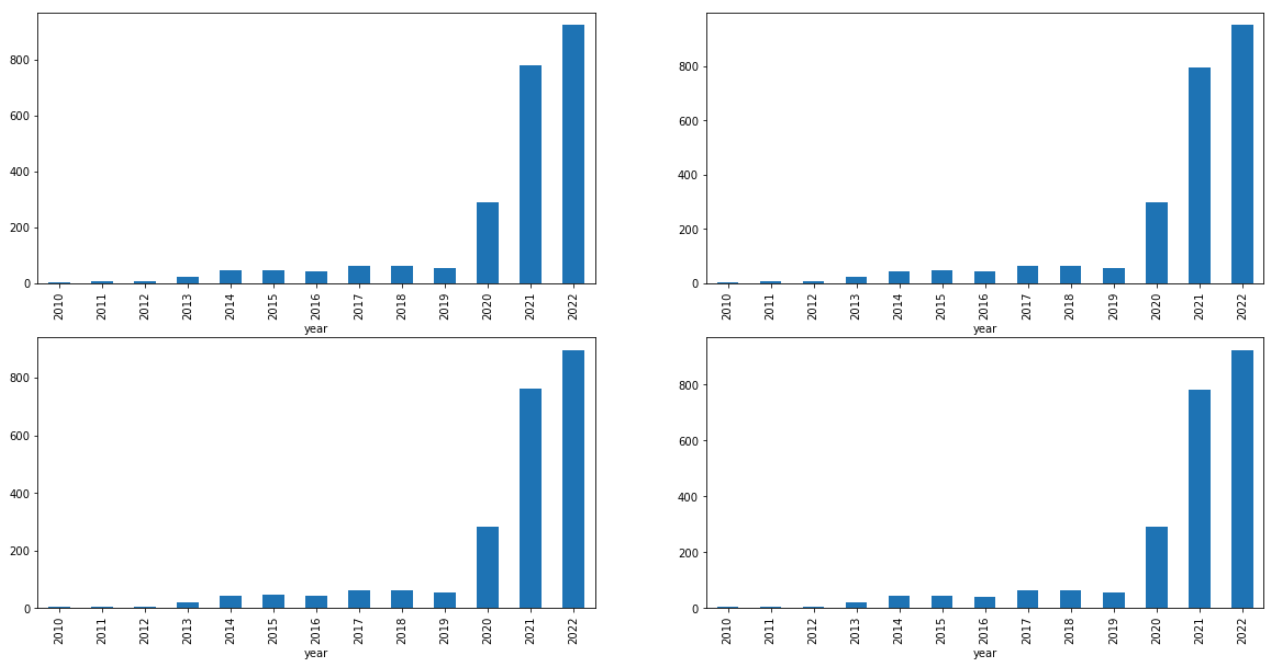
In [22]:
```
df[['Date','year','month','day']].head()
```

Out[22]:

	Date	year	month	day
0	2010-06-29	2010	6	29
1	2010-06-30	2010	6	30
2	2010-07-01	2010	7	1
3	2010-07-02	2010	7	2
4	2010-07-06	2010	7	6

In [23]:
```
data_grouped = df.groupby('year').mean()
```

In [24]:
```
data_grouped = df.drop('Date', axis=1).groupby('year').mean()
plt.subplots(figsize=(20,10))

for i, col in enumerate(['Open', 'High', 'Low', 'Close']):
  plt.subplot(2,2,i+1)
  data_grouped[col].plot.bar()
plt.show()
#I initially encountered incorrect time-based grouping due to improper date parsing, wh
```



In [25]:
```
df.drop('Date', axis=1).groupby('is_quarter_end').mean()
```

Out[25]:

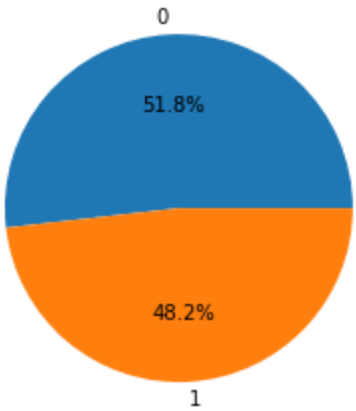
	Open	High	Low	Close	Volume	year	month	
is_quarter_end								
0	162.503657	166.006140	158.795106	162.640259	3.081474e+07	2016.004471	6.541977	15.
1	87.859608	90.038704	85.540369	87.790214	3.238128e+07	2015.535525	6.670201	15.

In [26]:

```
df['open-close']  = df['Open'] - df['Close']
df['low-high']  = df['Low'] - df['High']
df['target'] = np.where(df['Close'].shift(-1) > df['Close'], 1, 0)
```

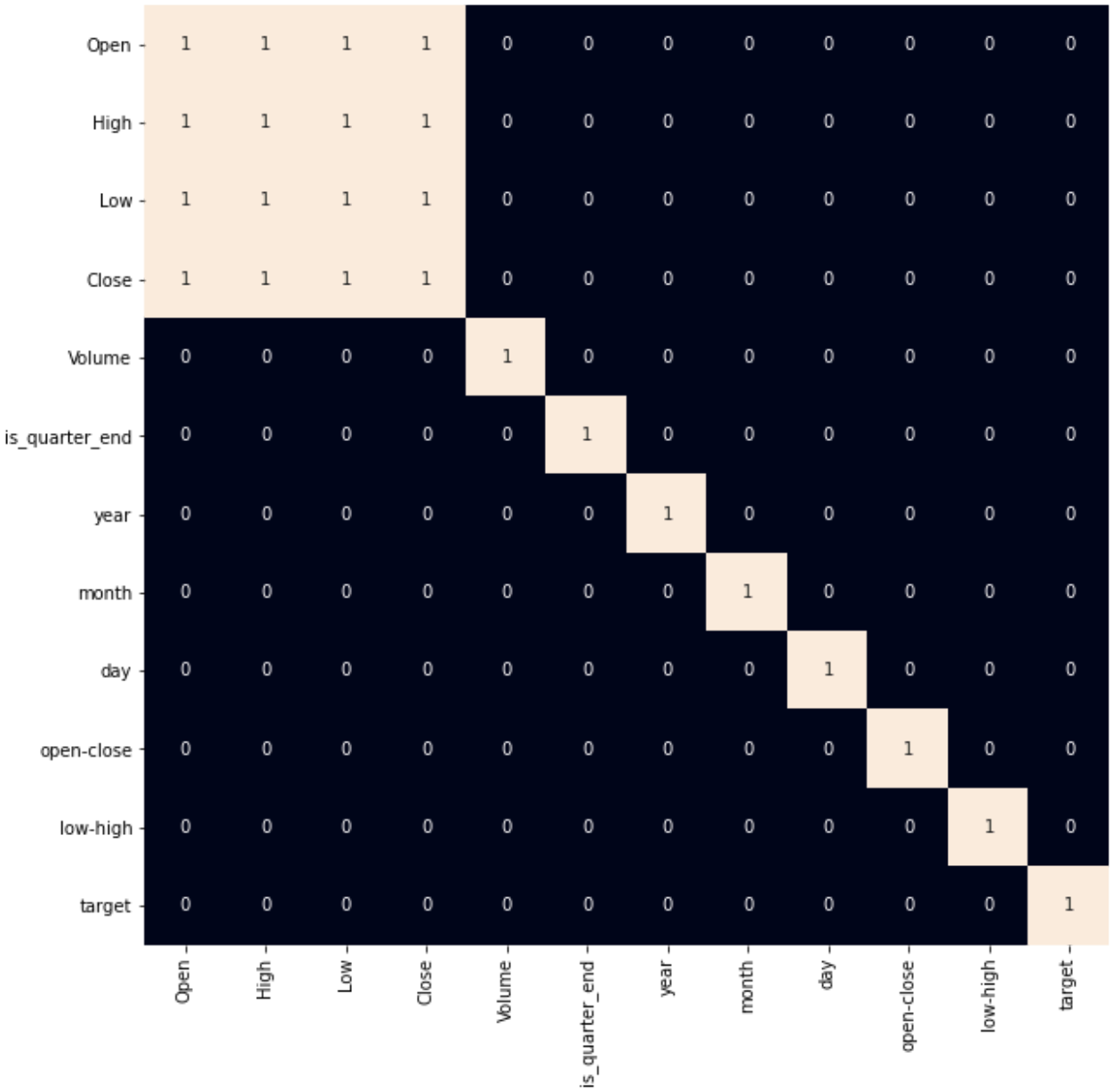
In [27]:

```
plt.pie(df['target'].value_counts().values,
        labels=[0, 1], autopct='%1.1f%%')
plt.show()
```



In [29]:

```
plt.figure(figsize=(10, 10))

# As our concern is with the highly
# correlated features only so, we will visualize
# our heatmap as per that criteria only.
sb.heatmap(df.drop('Date', axis=1).corr() > 0.9, annot=True, cbar=False)
plt.show()
```



In [30]:
```
features = df[['open-close', 'low-high', 'is_quarter_end']]
target = df['target']

scaler = StandardScaler()
features = scaler.fit_transform(features)

X_train, X_valid, Y_train, Y_valid = train_test_split(
    features, target, test_size=0.1, random_state=2022)
print(X_train.shape, X_valid.shape)
```

```
(2660, 3) (296, 3)
```

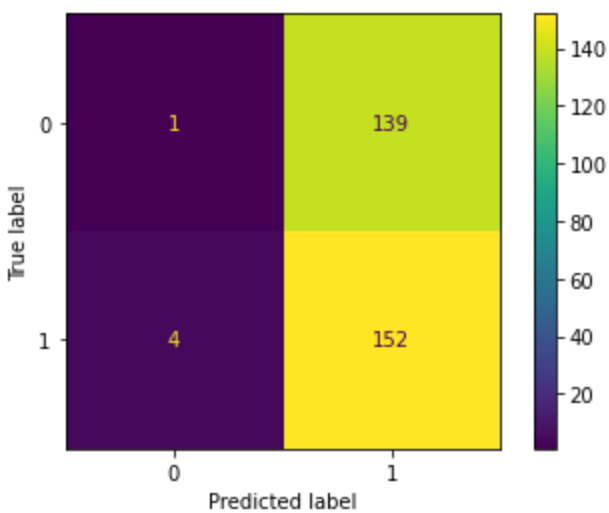
In [31]:
```
models = [LogisticRegression(), SVC(
  kernel='poly', probability=True), XGBClassifier()]

for i in range(3):
  models[i].fit(X_train, Y_train)

  print(f'{models[i]} : ')
  print('Training Accuracy : ', metrics.roc_auc_score(
    Y_train, models[i].predict_proba(X_train)[:,1]))
```

```
  print('Validation Accuracy : ', metrics.roc_auc_score(
    Y_valid, models[i].predict_proba(X_valid)[:,1]))
  print()
```

```
LogisticRegression() :
Training Accuracy :  0.5202201627166607
Validation Accuracy :  0.49793956043956045

SVC(kernel='poly', probability=True) :
Training Accuracy :  0.5031646268128758
Validation Accuracy :  0.5060897435897436

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
              num_parallel_tree=None, random_state=None, ...) :
Training Accuracy :  0.9103445348425894
Validation Accuracy :  0.5138507326007327
```

In [32]:

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(models[0], X_valid, Y_valid)
plt.show()
```



In [ ]:

```
#pred = model.predict(X_valid)
#from sklearn.linear_model import LinearRegression
#model = LinearRegression()
#model.fit(X_train, Y_train)
#X = df[['Open', 'High', 'Low', 'Volume']]
#Y = df['Close']
#pred = model.predict(X_valid)
#from sklearn.metrics import mean_squared_error, r2_score

#rmse = mean_squared_error(Y_valid, pred, squared=False)
#r2 = r2_score(Y_valid, pred)
```

```
#print("RMSE:", rmse)
#print("R2:", r2)
```